PART A: Concept Application

```
# Student Admission Decision System

# -------- INPUT --------
while True:
    try:
        entrance_score = int(input("Entrance Score (0-100): "))
        if 0 <= entrance_score <= 100:
            break
    except:
        pass
    print("Invalid entrance score.")

while True:
    try:
        gpa = float(input("GPA (0-10): "))
        if 0 <= gpa <= 10:
            break
    except:
        pass
    print("Invalid GPA.")

has_recommendation = input("Recommendation (yes/no): ").lower()
category = input("Category (general/obc/sc_st): ").lower()

while True:
    try:
        extracurricular_score = int(input("Extracurricular Score (0-10): "))
        if 0 <= extracurricular_score <= 10:
            break
    except:
        pass
    print("Invalid extracurricular score.")

# -------- AUTO MERIT --------
if entrance_score >= 95:
    print("\nResult: ADMITTED (Scholarship)")
    print("Reason: Entrance score ≥ 95 (Auto admit)")
else:

    # -------- BONUS --------
    bonus = 0
    if has_recommendation == "yes":
        bonus += 5
    if extracurricular_score > 8:
        bonus += 3

    effective_score = entrance_score + bonus

    # -------- CATEGORY CUT-OFF --------
    match category:
        case "general":
            cutoff = 75
        case "obc":
            cutoff = 65
        case "sc_st":
            cutoff = 55
        case _:
            print("Invalid category.")
            exit()
```

```
    # -------- DECISION --------
    if gpa < 7.0:
        print("\nResult: REJECTED")
        print("Reason: GPA below 7.0")

    elif effective_score >= cutoff:
        print(f"\nBonus Applied: {bonus}")
        print(f"Effective Score: {effective_score}")
        print("\nResult: ADMITTED (Regular)")
        print(f"Reason: Meets {category.upper()} cutoff")

    elif effective_score >= cutoff - 5:
        print("\nResult: WAITLISTED")
        print("Reason: Close to cutoff")

    else:
        print("\nResult: REJECTED")
        print("Reason: Score below cutoff")
```

```
Entrance Score (0-100): 72
GPA (0-10): 8.5
Recommendation (yes/no): yes
Category (general/obc/sc_st): general
Extracurricular Score (0-10): 9

Bonus Applied: 8
Effective Score: 80

Result: ADMITTED (Regular)
Reason: Meets GENERAL cutoff
```

PART B: Stretch Problem

```
# Indian Income Tax Calculator (New Regime FY 2024-25)

STANDARD_DEDUCTION = 75000

income = float(input("Enter Annual Income (₹): "))

# Apply standard deduction
taxable_income = max(0, income - STANDARD_DEDUCTION)

print(f"\nGross Income: ₹{income:,.0f}")
print(f"Standard Deduction: ₹{STANDARD_DEDUCTION:,.0f}")
print(f"Taxable Income: ₹{taxable_income:,.0f}\n")

# Tax slabs
slabs = [
    (300000, 0.00),
    (700000, 0.05),
    (1000000, 0.10),
    (1200000, 0.15),
    (1500000, 0.20),
    (float('inf'), 0.30)
]

previous_limit = 0
total_tax = 0

print("---- Tax Breakdown ----")

for limit, rate in slabs:
    if taxable_income > previous_limit:
```

```
        taxable_amount = min(taxable_income, limit) - previous_limit
        tax = taxable_amount * rate
        total_tax += tax

        print(f"₹{previous_limit:,.0f} – ₹{limit:,.0f} | "
              f"Income: ₹{taxable_amount:,.0f} | "
              f"Rate: {rate*100:.0f}% | "
              f"Tax: ₹{tax:,.0f}")

        previous_limit = limit

print("\n-----------------------")
print(f"Total Tax Payable: ₹{total_tax:,.0f}")

effective_rate = (total_tax / income) * 100 if income > 0 else 0
print(f"Effective Tax Rate: {effective_rate:.2f}%")
```

```
Enter Annual Income (₹): 1200000

Gross Income: ₹1,200,000
Standard Deduction: ₹75,000
Taxable Income: ₹1,125,000

---- Tax Breakdown ----
₹0 – ₹300,000 | Income: ₹300,000 | Rate: 0% | Tax: ₹0
₹300,000 – ₹700,000 | Income: ₹400,000 | Rate: 5% | Tax: ₹20,000
₹700,000 – ₹1,000,000 | Income: ₹300,000 | Rate: 10% | Tax: ₹30,000
₹1,000,000 – ₹1,200,000 | Income: ₹125,000 | Rate: 15% | Tax: ₹18,750

-----------------------
Total Tax Payable: ₹68,750
Effective Tax Rate: 5.73%
```

PART C: Interview Ready (20%) Q1: Conceptual Explain the difference between elif and multiple if statements.

1. elif (Else If)

elif is used in a single conditional chain. Once one condition becomes True, the remaining conditions in that chain are not evaluated.

Execution behavior:

```
1. Conditions are checked top → bottom

2. First True block executes

3. The rest are skipped
```

This ensures only one block runs.

2. Multiple if Statements

When using separate if statements, each condition is evaluated independently.

```
1. Execution behavior:

2. Every condition is checked

3. Multiple blocks can execute
```

```
# Example Where Output Differs
```

```
score = 85

if score >= 50:
    print("Pass")

if score >= 75:
    print("Distinction")
```

```
Pass
Distinction
```

```
score = 85

if score >= 75:
    print("Distinction")

elif score >= 50:
    print("Pass")
```

```
Distinction
```

```
def classify_triangle(a, b, c):

    # Check for invalid values
    if a <= 0 or b <= 0 or c <= 0:
        return "Invalid input: sides must be positive numbers"

    # Check triangle inequality condition
    if a >= b + c or b >= a + c or c >= a + b:
        return "Not a triangle"

    # Classification
    if a == b == c:
        return "Equilateral"

    elif a == b or b == c or a == c:
        return "Isosceles"

    else:
        return "Scalene"
```

```
print(classify_triangle(3,3,3))
print(classify_triangle(5,5,3))
print(classify_triangle(4,5,6))
print(classify_triangle(1,2,3))
print(classify_triangle(-2,4,5))
```

```
Equilateral
Isosceles
Scalene
Not a triangle
Invalid input: sides must be positive numbers
```

```
score = 85

if score >= 60:
    grade = 'D'
if score >= 70:
    grade = 'C'
if score >= 80:
    grade = 'B'
if score >= 90:
    grade = 'A'
```

```
print(grade)
```

```
B
```

Why This Fix Works

1. elif ensures only one condition executes
2. Conditions are checked from highest grade to lowest
3. Once a match is found, Python skips the remaining checks
4. This guarantees the correct grade classification.

```
score = 85

if score >= 90:
    grade = 'A'
elif score >= 80:
    grade = 'B'
elif score >= 70:
    grade = 'C'
elif score >= 60:
    grade = 'D'
else:
    grade = 'F'

print(grade)
```

```
B
```

PART D: AI-Augmented Task

Write a Python program that validates an Indian PAN card number format using if-else conditions. PAN format: 5 uppercase letters, 4 digits, 1 uppercase letter (e.g., ABCDE1234F). The 4th character indicates the type of taxpayer.

```
def validate_pan(pan):
    if len(pan) != 10:
        return "Invalid PAN length"

    if not pan[:5].isalpha():
        return "First 5 characters must be letters"

    if not pan[5:9].isdigit():
        return "Next 4 characters must be digits"

    if not pan[9].isalpha():
        return "Last character must be a letter"

    taxpayer_type = pan[3]

    return f"Valid PAN. Taxpayer type indicator: {taxpayer_type}"


pan = input("Enter PAN number: ")

if pan.isupper():
    print(validate_pan(pan))
else:
```

```
        print("PAN must contain uppercase letters only")
```

```
Enter PAN number: ABCDE1234@
Last character must be a letter
```

Critical Evaluation Are all positions validated correctly?

Partially correct.

Correct validations:

```
Length check (len == 10)

First 5 characters are letters

Next 4 characters are digits

Last character is a letter
```

Problems:

It checks .isalpha() but does not guarantee uppercase letters inside the function.

The isupper() check outside the function is not reliable, because it only verifies that existing letters are uppercase but does not check digits properly.

Example issue:

ABCDE1234f

Lowercase last letter would bypass internal validation if outer check was removed.

Are edge cases handled?

Some edge cases are missing:

- Lowercase letters Not validated properly inside function
- Special characters Not explicitly checked
- Incorrect taxpayer code Not validated
- Spaces in input Not handled

Example failing cases:

ABCDE1234@

ABC1E1234F

ABCDE12345

Is the approach correct?

The prompt asked for if-else conditions, so a character-by-character approach is appropriate.

Alternative approach:

```
Regex validation
```

Example regex:

```
[A-Z]{5}[0-9]{4}[A-Z]
```

Regex is shorter and more robust, but the assignment requirement suggests manual validation, so the current approach is acceptable for learning purposes.

Is the code Pythonic?

```
Not fully Pythonic.
```

Issues:

```
1. Validation logic split between main code and function

2. Uppercase check done outside the function

3. Error messages could be clearer

4. No check for specific character positions
```

A Pythonic design would:

```
1. Keep all validation inside the function

2. Use clear logical steps

3. Validate each position explicitly
```

```
def validate_pan(pan):

    if len(pan) != 10:
        return "Invalid PAN: Must be exactly 10 characters"

    # First 5 characters must be uppercase letters
    for i in range(5):
        if not ('A' <= pan[i] <= 'Z'):
            return "Invalid PAN: First 5 characters must be uppercase letters"

    # Next 4 characters must be digits
    for i in range(5, 9):
        if not pan[i].isdigit():
            return "Invalid PAN: Characters 6–9 must be digits"

    # Last character must be uppercase letter
    if not ('A' <= pan[9] <= 'Z'):
        return "Invalid PAN: Last character must be an uppercase letter"

    taxpayer_type = pan[3]

    return f"Valid PAN. Taxpayer type indicator: {taxpayer_type}"


pan = input("Enter PAN number: ").strip()

result = validate_pan(pan)
print(result)
```

```
Enter PAN number: ABCDE1234f
Invalid PAN: Last character must be an uppercase letter
```

PART E: Bonus Challenge

```
# transaction_validator.py

amount = float(input("Transaction amount (Rs): "))
category = input("Category (food/travel/electronics/other): ").lower()
hour = int(input("Hour of transaction (0-23): "))
daily_spent = float(input("Amount already spent today (Rs): "))
vip = input("VIP customer? (yes/no): ").lower()

# --- Limits ---
single_limit = 50000 * (2 if vip == "yes" else 1)
daily_limit = 100000 * (2 if vip == "yes" else 1)

food_limit = 5000 * (2 if vip == "yes" else 1)
electronics_limit = 30000 * (2 if vip == "yes" else 1)

# --- Decision Logic ---

# BLOCK rules
if amount > single_limit:
    print("BLOCKED: exceeds single transaction limit")

elif daily_spent + amount > daily_limit:
    print("BLOCKED: exceeds daily spending limit")

# FLAG rules
elif hour < 6 or hour > 23:
    print("FLAGGED: unusual transaction time")

# Category rules
elif category == "food" and amount > food_limit:
    print("FLAGGED: food transaction unusually high")

elif category == "electronics" and amount > electronics_limit:
    print("FLAGGED: electronics transaction unusually high")

else:
    print("APPROVED")
```

```
Transaction amount (Rs): 3000
Category (food/travel/electronics/other): food
Hour of transaction (0-23): 14
Amount already spent today (Rs): 10000
VIP customer? (yes/no): no
APPROVED
```

Start coding or generate with AI.